

LOG IN



opensource.com



ARTICLES

RESOURCES ▾

DOWNLOADS

FREQUENTLY ASKED QUESTIONS

Using Bash traps in your scripts

Traps help your scripts end cleanly, whether they run successfully or not.

By [Seth Kenlon](#) (Team, Red Hat)

June 29, 2020 | [13 Comments](#) | 7 min read

126 readers like this.



Image by: WOCinTech Chat. Modified by Opensource.com. CC BY-SA 4.0

It's easy to detect when a shell script starts, but it's not always easy to know when it stops. A script might end normally, just as its author intends it to end, but it could also fail due to an unexpected fatal error. Sometimes it's beneficial to preserve the remnants of whatever was in progress when a script failed, and other times it's inconvenient. Either way, detecting the end of a script and reacting to it in some pre-calculated manner is why the [Bash](#) `trap` directive exists.

Responding to failure

Here's an example of how one failure in a script can lead to future failures. Say you have written a program that creates a temporary directory in `/tmp` so that it can unarchive and process files before bundling them back together in a different format:

```
#!/usr/bin/env bash
CWD=`pwd`
TMP=${TMP:-/tmp/tmpdir}

## create tmp dir
mkdir "${TMP}"

## extract files to tmp
tar xf "${1}" --directory "${TMP}"

## move to tmpdir and run commands
pushd "${TMP}"
for IMG in *.jpg; do
    mogrify -verbose -flip -flop "${IMG}"
done
tar --create --file "${1%.*}.tar" *.jpg

## move back to origin
popd

## bundle with bzip2
bzip2 --compress "${TMP}/${1%.*}.tar" \
    --stdout > "${1%.*}.tbz"

## clean up
/usr/bin/rm -r /tmp/tmpdir
```

Most of the time, the script works as expected. However, if you accidentally run it on an archive filled with PNG files instead of the expected JPEG files, it fails halfway through. One failure leads to another, and eventually, the script exits without reaching its final directive to remove the temporary directory. As long as you manually remove the directory, you can recover quickly, but if you aren't around to do that, then the next time the script runs, it has to deal with an existing temporary directory full of unpredictable leftover files.

One way to combat this is to reverse and double-up on the logic by adding a precautionary removal to the start of the script. While valid, that relies on brute force instead of structure. A more elegant solution is `trap`.

Catching signals with trap

The `trap` keyword catches *signals* that may happen during execution. You've used one of these signals if you've ever used the `kill` or `killall` commands, which call `SIGTERM` by default. There are many other signals that shells respond to, and you can see most of them with `trap -l` (as in "list"):

```
$ trap --list
1) SIGHUP          2) SIGINT          3) SIGQUIT         4) SIGILL          5) SIGTRAP
6) SIGABRT         7) SIGBUS         8) SIGFPE          9) SIGKILL         10) SIGUSR1
11) SIGSEGV        12) SIGUSR2        13) SIGPIPE        14) SIGALRM        15) SIGTERM
16) SIGSTKFLT      17) SIGCHLD        18) SIGCONT        19) SIGSTOP        20) SIGTSTP
21) SIGTTIN        22) SIGTTOU        23) SIGURG         24) SIGXCPU        25) SIGXFSZ
26) SIGVTALRM      27) SIGPROF        28) SIGWINCH       29) SIGIO          30) SIGPWR
31) SIGSYS         34) SIGRTMIN       35) SIGRTMIN+1     36) SIGRTMIN+2     37) SIGRTMIN+3
38) SIGRTMIN+4     39) SIGRTMIN+5     40) SIGRTMIN+6     41) SIGRTMIN+7     42) SIGRTMIN+8
43) SIGRTMIN+9     44) SIGRTMIN+10    45) SIGRTMIN+11    46) SIGRTMIN+12    47) SIGRTMIN+13
48) SIGRTMIN+14    49) SIGRTMIN+15    50) SIGRTMAX-14    51) SIGRTMAX-13    52) SIGRTMAX-12
53) SIGRTMAX-11    54) SIGRTMAX-10    55) SIGRTMAX-9     56) SIGRTMAX-8     57) SIGRTMAX-7
58) SIGRTMAX-6     59) SIGRTMAX-5     60) SIGRTMAX-4     61) SIGRTMAX-3     62) SIGRTMAX-2
63) SIGRTMAX-1     64) SIGRTMAX
```

Any of these signals may be anticipated with `trap`. In addition to these, `trap` recognizes:

- `EXIT`: Occurs when the shell process itself exits
- `ERR`: Occurs when a command (such as `tar` or `mkdir`) or a built-in command (such as `pushd` or `cd`)

completes with a non-zero status

- **DEBUG**: A Boolean representing debug mode

To set a trap in Bash, use **trap** followed by a list of commands you want to be executed, followed by a list of signals to trigger it.

For instance, this trap detects a **SIGINT**, the signal sent when a user presses **Ctrl+C** while a process is running:

```
trap "{ echo 'Terminated with Ctrl+C'; }" SIGINT
```

The example script with temporary directory problems can be fixed with a trap detecting **SIGINT**, errors, and successful exits:

```
#!/usr/bin/env bash
CWD=`pwd`
TMP=${TMP:-/tmp/tmpdir}

trap \
"{ /usr/bin/rm -r "${TMP}" ; exit 255; }" \
SIGINT SIGTERM ERR EXIT

## create tmp dir
mkdir "${TMP}"
tar xf "${1}" --directory "${TMP}"

## move to tmp and run commands
pushd "${TMP}"
for IMG in *.jpg; do
    mogrify -verbose -flip -flop "${IMG}"
done
tar --create --file "${1%.*}.tar" *.jpg

## move back to origin
popd

## zip tar
bzip2 --compress $TMP/"${1%.*}.tar" \
--stdout > "${1%.*}.tbz"
```

For complex actions, you can simplify **trap** statements with [Bash functions](#).

Traps in Bash

Traps are useful to ensure that your scripts end cleanly, whether they run successfully or not. It's never safe to rely completely on automated garbage collection, so this is a good habit to get into in general. Try using them in your scripts, and see what they can do!

What to read next

More on Bash

- [Bash cheat sheet](#)
- [An introduction to programming with Bash](#)
- [A sysadmin's guide to Bash scripting](#)
- [Latest Bash articles](#)



How to write functions in Bash

Reduce redundancy and maintenance in your code by writing functions.



[Seth Kenlon](#) (Team, Red Hat)



How to program with Bash: Syntax and tools

Learn basic Bash programming syntax and tools, as well as how to use variables and control operators, in the first article in this three-part series.



[David Both](#) (Correspondent)



5 ways to improve your Bash scripts

Find out how Bash can help you tackle the most challenging tasks.



[Alan Formy-Duval](#) (Correspondent)

Tags:

[BASH](#)

[SCRIPTING](#)



Seth Kenlon

Seth Kenlon is a UNIX geek, free culture advocate, independent multimedia artist, and D&D nerd. He has worked in the film and computing industry, often at the same time.

[More about me](#)

13 Comments

These comments are closed.



[ersen](#) | June 29, 2020 No readers like this yet.

I tried EXIT trap, it also executed when the script is killed with HUP, TERM and INT signals. I think it executes whenever the script exits, whether with 0 exit status or not.



[Seth Kenlon](#) | June 29, 2020 No readers like this yet.

I've clarified the article for future readers.



Erik Lundmark | June 29, 2020 No readers like this yet.

I think newer versions of the trap command use ``-l`` instead of ``--list`` - I'm on Debian Testing



[Seth Kenlon](#) | June 29, 2020 No readers like this yet.

I see that on RHEL 8, too. Updated, thanks!



[01101001b](#) | June 30, 2020 No readers like this yet.

I found this article VERY useful and well explained. Thank you so much!



[Seth Kenlon](#) | June 30, 2020 No readers like this yet.

I'm glad this is useful to you, 01101001b!



Tomi-1234 | July 1, 2020 No readers like this yet.

good stuff, but nitpicks:

change ``pwd`` to `$PWD`, works w/o subshell

quote uses of `"$TMP"` -- someone could have `$IFS` characters there

ditto for `$IMG`

(note that in cases of `var=$TMP` quotes not needed)

`trap "/usr/bin/rm -r "$TMP"; exit 255" SIGINT SIGTERM ERR EXIT`

should work. have you checked trap is not executed twice (e.g. for ERR and for EXIT)?

(and last (but sure not least), I personally recommend `set -euf` to all shell scripts)



[Seth Kenlon](#) | July 2, 2020 No readers like this yet.

I agree with many of these suggestions, and have altered some parts of the article accordingly. Other points are well made, but I'll leave them as comments for users to decide upon for themselves.



eli | July 2, 2020 No readers like this yet.

When trapping multiple signals (e.g., `SIGINT SIGTERM ERR EXIT`), is there a way to exit with a code corresponding to the trap? Thanks



[Seth Kenlon](#) | July 3, 2020 No readers like this yet.

The only way I know is to create a trap statement for each one.

Place all of your traps at the start of your script to ensure Bash is "aware" of all the different conditions.

I'd be interested in knowing if someone has a fancier way of doing it (although I'd also want to test it rigourously before committing to using it).



[sarlacii](#) | July 14, 2020 No readers like this yet.

Hi there, thought to mention that brackets are preferred over backticks for command substitution, as backticks are deprecated. See "<https://mywiki.woledge.org/BashFAQ/082>".



[Steven Bakker](#) | July 15, 2020 No readers like this yet.

In the description of ERR and EXIT, it may be helpful to clarify that ERR gets triggered when a command (either external like "mkdir", "tar", or internal like "pushd") completes with an error status.

EXIT gets triggered when the shell process (script) itself exits.



[Seth Kenlon](#) | July 15, 2020 No readers like this yet.

Great clarification. I've added this to the article text for posterity.

Thanks for the feedback!

Related Content



[Make a web-safe color guide with Bash](#)



[How I configure Vim as my default editor on Linux](#)



[How I dynamically generate Jekyll config files](#)



This work is licensed under a Creative Commons Attribution-Share Alike 4.0 International License.

ABOUT THIS SITE

opensource.com

The opinions expressed on this website are those of each author, not of the author's employer or of Red Hat.

Opensource.com aspires to publish all content under a **Creative Commons license** but may not be able to do so in all cases. You are responsible for ensuring that you have the necessary permission to reuse any work on this site. Red Hat and the Red Hat logo are trademarks of Red Hat, LLC, registered in the United States and other countries.

A note on advertising: Opensource.com does not sell advertising on the site or in any of its newsletters.

Copyright ©2026 Red Hat, LLC

[Privacy Policy](#) | [Terms of use](#) |